

HOME DINÁMICO

Arquitectura, administración y renderizado

Moira Bikinis · Junio 2026

Este documento describe la implementación del home dinámico de Moira: cómo está estructurado, cómo lo edita el administrador desde el panel Filament, y cómo se renderiza en el frontend Next.js.

1. Contexto y objetivo

El home anterior de Moira era estático: cargaba la primera categoría y 4 productos aleatorios destacados, sin posibilidad de edición desde el admin. El objetivo fue convertirlo en un home completamente editable, similar a como funciona el Page Builder de Magento 2 o los widgets de WordPress, pero más simple y sin necesidad de un editor visual drag-and-drop.

La referencia visual fue el tema Avanam (<http://localhost:8090/>), que tiene:

- Una galería/slider full-width con fotos lifestyle de la tienda
- Un banner promocional con imagen + texto superpuesto
- Una sección "TOP PRODUCT" con tabs: FEATURED, LATEST, SPECIAL, BEST SELLER
- Cada tab muestra un carrusel de 4 productos

2. Arquitectura del sistema

El sistema tiene tres capas: base de datos, API Laravel, y frontend Next.js.

CAPA	TECNOLOGÍA	RESPONSABILIDAD
Base de datos	PostgreSQL	Tabla home_sections con settings JSON
API	Laravel 13 + Filament	CRUD admin + endpoint GET /api/v1/home
Frontend	Next.js 16	Renderiza secciones según su tipo

Flujo de datos

Paso	Actor	Acción
1	Admin edita en Filament	Agrega/edita/reordena secciones en /admin
2	Filament guarda en BD	Tabla home_sections, settings como JSON
3	Frontend solicita	GET /api/v1/home (sin auth, público)
4	Laravel resuelve	Ejecuta queries por tipo de tab, convierte paths de imagen
5	Next.js renderiza	Elige el componente React según section.type

3. Base de datos — tabla home_sections

La tabla es intencionalmente simple. Toda la configuración específica por tipo se almacena en la columna settings como JSON, evitando columnas nulas por tipo.

Columna	Tipo	Descripción
---------	------	-------------

id	bigint PK	Identificador
type	varchar	hero_slider product_tabs banner
title	varchar null	Título visible de la sección (ej: TOP PRODUCT)
settings	json	Configuración completa según el tipo (ver abajo)
sort_order	smallint	Orden de aparición en el home (menor = primero)
is_active	boolean	Si false, la sección no aparece en el frontend
created_at	timestamp	
updated_at	timestamp	

Estructura del JSON según tipo

hero_slider

```
{ "slides": [ { "image": "home-slides/foto.webp", "title": "Nueva colección", "subtitle": "Envío gratis", "button_text": "Ver", "button_link": "/categories/bikinis" } ] }
```

product_tabs

```
{ "per_tab": 8, "tabs": [ { "label": "NOVEDADES", "type": "latest" }, { "label": "OFERTAS", "type": "on_sale" }, { "label": "BIKINIS", "type": "by_category", "category_slug": "bikinis" }, { "label": "ESPECIAL", "type": "custom", "product_ids": [12, 34, 7] } ] }
```

banner

```
{ "image": "home-banners/promo.webp", "title": "Temporada verano", "subtitle": "Hasta 50% off", "button_text": "Ver ofertas", "button_link": "/categories/ofertas" }
```

4. Tipos de sección

■ hero_slider — Galería / Slider

Banner full-width con múltiples slides. Soporta imagen de fondo, título, subtítulo y botón CTA con link. Los slides se reordenan con drag & drop. El frontend hace auto-avance cada 5 segundos y tiene flechas de navegación y dots indicadores.

Campo	Descripción
image	Imagen del slide (WebP auto-convertido). Requerida.
title	Título superpuesto. Opcional.
subtitle	Subtítulo. Opcional.
button_text	Texto del botón CTA. Opcional.
button_link	URL de destino del botón. Opcional.

■ product_tabs — Tabs de productos

Sección con múltiples tabs, cada uno mostrando una grilla de productos. El admin configura cuántos tabs mostrar, en qué orden, cómo se llaman y qué productos carga cada uno. El frontend renderiza tabs clicables; la grilla de productos del tab activo se muestra debajo.

Campo	Descripción
label	Etiqueta del tab (texto del botón, ej: NOVEDADES).
type	Fuente de productos (ver sección 5).
category_slug	Solo para type=by_category. Slug de la categoría a mostrar.
product_ids	Solo para type=custom. IDs de los productos elegidos manualmente.

■ banner — Banner promocional

Imagen de fondo con overlay oscuro, texto superpuesto y botón CTA. Ideal para campañas de temporada, liquidaciones o novedades.

Campo	Descripción
image	Imagen del banner (WebP auto-convertido).
title	Título superpuesto.
subtitle	Subtítulo o descripción breve.
button_text	Texto del botón.
button_link	URL de destino.

5. Fuentes de productos para tabs

Cada tab del tipo `product_tabs` tiene una 'fuente' que determina qué query ejecuta el backend. El admin no necesita conocer SQL — elige una opción de un select.

Fuente (type)	Label en admin	Query ejecutada	Fallback
latest	Más recientes	ORDER BY created_at DESC LIMIT n	—
on_sale	En oferta	WHERE sale_price IS NOT NULL ORDER BY created_at DESC	—
best_sellers	Más vendidos	JOIN order_items GROUP BY product_id ORDER BY SUM(qty)	Si no hay órdenes: más recientes
by_category	Por categoría	JOIN category_product WHERE slug = ?	—
custom	Personalizado	WHERE id IN (product_ids[])	—

El campo `per_tab` (default: 8) controla cuántos productos se devuelven por tab. Se aplica a todas las fuentes automáticas; en custom se muestran todos los elegidos.

6. Panel de administración — Filament

La edición del home se hace desde:

→ <http://localhost:8080/admin> → Marketing → Home

Vista lista

Muestra todas las secciones ordenadas por `sort_order`. Desde aquí se puede:

- Reordenar arrastrando filas (drag & drop sobre la columna de orden)
- Ver el tipo, título y estado de cada sección
- Entrar a editar una sección
- Eliminar secciones

Formulario de edición

El formulario es dinámico: cambia según el tipo seleccionado.

Galería / Slider (`hero_slider`)

- Tipo: selector fijo
- Slides: Repeater con drag & drop — agregar/eliminar/reordenar
- Por slide: subir imagen (→ auto-convertida a WebP), título, subtítulo, texto botón, link
- Orden de sección: número
- Sección activa: toggle

Tabs de productos (`product_tabs`)

- Tipo: selector
- Título: texto libre (ej: 'TOP PRODUCT')
- Productos por tab: número (default 8)
- Tabs: Repeater — agregar/eliminar/reordenar tabs
- Por tab: etiqueta (texto libre), fuente (select), categoría o productos según fuente
- Orden de sección y estado activo

Banner (`banner`)

- Tipo: selector
- Imagen: upload (auto-convertida a WebP)
- Título, subtítulo, texto del botón, link del botón
- Orden de sección y estado activo

Localización del código Filament

Archivo	Responsabilidad
HomeSectionResource.php	Registro del resource, pack/unpack settings JSON
HomeSectionForm.php	Formulario dinámico con Repeaters y campos condicionales
HomeSectionsTable.php	Lista con reorder, badges de tipo, toggle activo
CreateHomeSection.php	Página de creación con mutateFormDataBeforeSave
EditHomeSection.php	Edición con mutateFormDataBeforeFill + BeforeSave

7. API Laravel — HomeController

Endpoint público (sin autenticación):

```
GET /api/v1/home
```

Retorna todas las secciones activas ordenadas por sort_order. Para secciones de tipo product_tabs, el controller ejecuta las queries de productos y los embebe directamente en la respuesta (no hay lazy loading). Las rutas de imagen se convierten a URLs absolutas con Storage::url().

Respuesta de ejemplo

```
{ "data": [ { "id": 1, "type": "hero_slider", "title": null, "sort_order": 0, "settings": {  
  "slides": [ { "image": "http://...storage/home-slides/foto.webp", "title": "Nueva colección",  
    "button_text": "Ver", "button_link": "/categories/bikinis" } ] }, { "id": 2, "type":  
  "product_tabs", "title": "TOP PRODUCT", "sort_order": 1, "settings": { "per_tab": 8, "tabs": [  
    { "label": "NOVEDADES", "type": "latest", "products": [ { "id": 5, "name": "Bikini rayado",  
      "price": 12500, ... } ] } ] } ] } ] }
```

Archivo

```
moira-api/app/Http/Controllers/Api/V1/HomeController.php
```

8. Frontend Next.js

Estructura de archivos

Archivo	Tipo	Responsabilidad
src/app/page.tsx	Server	Llama a api.getHome(), itera secciones y renderiza el componente correcto
src/components/home/HeroSlider.tsx	Client	Slider con flechas, dots y auto-avance. Gestiona estado del slide activo
src/components/home/ProductTabsSection.tsx	Client	Tabs clicables y grilla de productos. Gestiona estado del tab activo
src/components/home/BannerSection.tsx	Server	Banner estático con imagen, texto y CTA
src/lib/api.ts	—	Método getHome() y tipos TypeScript: HomeSection, HeroSlide, ProductTab
src/app/globals.css	—	Estilos: .hero-slider, .home-tabs-section, .home-banner

Lógica de renderizado en page.tsx

```
sections.map(section => { if (section.type === "hero_slider") → if (section.type === "product_tabs") → if (section.type === "banner") → })
```

Tipos TypeScript (src/lib/api.ts)

```
type HeroSlide = { image, title, subtitle, button_text, button_link } type ProductTab = { label, type, category_slug?, product_ids?, products: Product[] } type HomeSection = HomeSectionHeroSlider | HomeSectionProductTabs | HomeSectionBanner
```

Fallback sin secciones

Si la API devuelve un array vacío (ej: primera vez, sin configurar), el home muestra un mensaje: 'Configurará las secciones del home desde el panel de administración.' Nunca hay pantalla en blanco.

9. Cómo agregar un nuevo tipo de sección

El sistema está diseñado para ser extensible. Agregar un tipo nuevo como 'countdown' o 'instagram_feed' requiere 4 pasos:

Paso 1: BD: ningún cambio necesario

La columna settings JSON absorbe cualquier estructura nueva.

Paso 2: API: HomeController.php

Agregar un if para el nuevo type en resolveSection(). Opcionalmente agregar lógica de query en resolveTabProducts().

Paso 3: Admin: HomeSectionForm.php

Agregar el type al Select de opciones. Agregar una Section con los campos correspondiente, visible(fn => \$get('type')) === 'nuevo_tipo'). Agregar pack/unpack en HomeSectionResource.

Paso 4: Frontend: page.tsx

Agregar un nuevo componente React en src/components/home/. Agregar el case en el switch de page.tsx.

10. Decisiones de diseño

■ ¿Por qué una sola tabla con JSON?

Evita tener que hacer JOIN entre tablas de tipo slide y tabs. El JSON es flexible y Eloquent lo castea automáticamente a array. La contrapartida es que no se puede hacer queries SQL sobre el contenido del JSON, pero no es necesario para este caso.

■ ¿Por qué los productos se embeben en la respuesta?

Evita que el frontend haga N pedidos (uno por tab). Al cargar la página, todos los productos ya están disponibles. El tab switching es instantáneo. La desventaja es una respuesta más grande, pero aceptable para 8 productos × 4 tabs.

■ ¿Por qué el slider usa estado local (no URL)?

El slide actual no es información que necesite persistir. URL params agregarían complejidad innecesaria y romperían el scroll de la página.

■ ¿Por qué no hay 'is_featured' en Product?

El tab FEATURED del tema Avanam es básicamente el mismo que Más Vendidos o Por Categoría. En lugar de agregar un flag booleano a los productos (que habría que mantener manualmente), el tipo 'custom' permite al admin armar exactamente la lista que quiere.

■ ¿Por qué pack/unpack settings en mutateFormDataBeforeSave?

Filament no puede mapear directamente campos de un Repeater a sub-claves de un JSON cuando hay múltiples tipos de sección con estructuras diferentes. El pack/unpack en las páginas Create/Edit garantiza que el JSON siempre esté limpio y solo tenga las claves relevantes para el tipo activo.

Documento generado automáticamente por Claude Code · Moira Bikinis · Junio 2026